

FileEditSelectionViewGoRunTerminalHelpPython\_Test

EXPLORER

PYTHON\_TEST

alignment\_result.png

arithmetic.py

beach.png

bitwise.py

black.jpg

checkerboard.png

color\_splash.png

composite\_masterpiece.png

create\_test\_image\_day3.py

create\_test\_image.py

crop.py

custom\_mask.png

day5\_transformation.py

day6\_arithmetic\_bitwise.py

dissolve.mp4

drawin\_basics.py

drawing.py

ex\_1\_day\_3.py

ex\_2\_day\_3.py

ex\_3\_day\_3.py

ex\_4\_day\_3.py

exercise\_1\_house.py

exercise\_1.py

exercise5\_heart\_mask.py

splitting\_merging.py

masking.py

bitwise.py

arithmetic.py

gradient.py

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

arithmetic.py

1# USAGE

2# python arithmetic.py --image ../images/trex.png

3

4# Import the necessary packages

5from \_\_future\_\_ import print\_function

6import numpy as np

7import argparse

8import cv2

9

10# Construct the argument parser and parse the arguments

11ap = argparse.ArgumentParser()

12ap.add\_argument("-i", "--image", required = True,

13| help = "Path to the image")

14args = vars(ap.parse\_args())

15

16# Load the image and show it

17image = cv2.imread(args["image"])

18cv2.imshow("Original", image)

19

20# Images are NumPy arrays, stored as unsigned 8 bit integers.

21# What does this mean? It means that the values of our pixels

22# will be in the range [0, 255]. When using functions like

23# cv2.add and cv2.subtract, values will be clipped to this

24# range, even if the added or subtracted values fall outside

PROBLEMS

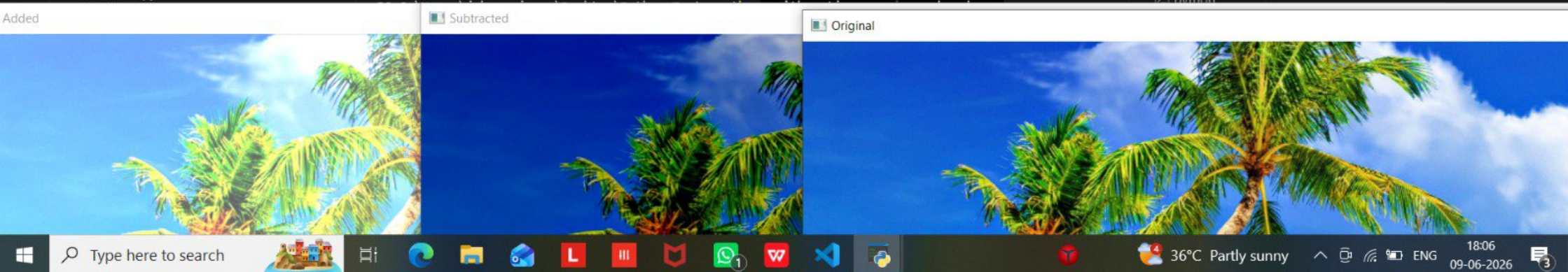
OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

python



File Edit Selection View Go Run Terminal Help Python\_Test

Sign In

CHAT

Rectangle

Circle

```
28 cv2.waitKey(0)
29
30 # A bitwise 'OR' examines every pixel in rectangle
31 # and circle. If EITHER pixel in rectangle or circle
32 # is greater than zero, then the output pixel has a
33 # value of 255, otherwise it is 0.
34 bitwiseOr = cv2.bitwise_or(rectangle, circle)
35 cv2.imshow("OR", bitwiseOr)
36 cv2.waitKey(0)
37
38 # The bitwise 'XOR' is identical to the 'OR' function,
39 # with one exception: both rectangle and circle are
40 # not allowed to have values greater than 0.
41 bitwiseXor = cv2.bitwise_xor(rectangle, circle)
42 cv2.imshow("XOR", bitwiseXor)
43 cv2.waitKey(0)
44
45 # Finally, the bitwise 'NOT' inverts the values of
46 # the pixels. Pixels with a value of 255 become 0,
47 # and pixels with a value of 0 become 255.
48 bitwiseNot = cv2.bitwise_not(circle)
49 cv2.imshow("NOT", bitwiseNot)
50 cv2.waitKey(0)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\jebtron\jason\Desktop\Python\_Test> python bitwise.py

powerShell python python python python python python python python python

Describe what to build

Local Default Approvals

Ln 45, Col 51 Spaces: 4 UTF-8 CRLF Python

Type here to search

This golden retriever...

18:03 09-06-2026

FileEditSelectionViewGoRunTerminalHelpPython\_Test

EXPLORER

PYTHON\_TEST

getting\_and\_setting.py

gradient.png

heart\_blend.png

house.png

imutils.cpython-314.pyc

imutils.py

load\_display\_save.py

masked\_result.png

masking.py

my\_masterpiece.png

output.bmp

output.jpg

output.tiff

panorama\_prep.png

pixel\_basics.py

red.jpg

resize.py

rotate.py

smart\_crop.png

splitting\_merging.py

test\_gradient.png

test\_image\_gray.png

test\_image.png

test\_pattern.png

thumbnail\_gallery.png

translation.py

trex.png

verify\_install.py

white.jpg

OUTLINE

TIMELINE

exercise5\_heart\_mask.py

splitting\_merging.py

masking.py

gradient.png

getting\_and\_setting.py

masking.py

```
1 # USAGE
2 # python masking.py --image ../images/beach.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import cv2
8
9 # Construct the argument parser and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-i", "--image", required = True,
12               help = "Path to the image")
13 args = vars(ap.parse_args())
14
15 # Load the image and show it
16 image = cv2.imread(args["image"])
17 cv2.imshow("Original", image)
18
19 # Masking allows us to focus only on parts of an image that
20 # interest us. A mask is the same size as our image, but has
21 # only two pixel values, 0 and 255. Pixels with a value of 0
22 # are ignored in the original image, and mask pixels with a
23 # value of 255 are allowed to be kept. For example, let's
24 # construct a mask with a 150x150 square at the center of it
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\jebtron\_jason\Desktop\Python\_Test> python masking.py --image beach.png

powershell

powershell

python

python

powershell

powershell

powershell

python

python

python

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

+ masking.py

Describe what to build

+ | | Auto |

↑

Local

Default Approvals

Ln 13, Col 29

Spaces: 4

UTF-8

CRLF

{ } Python

Record high

18:02

09-06-2026



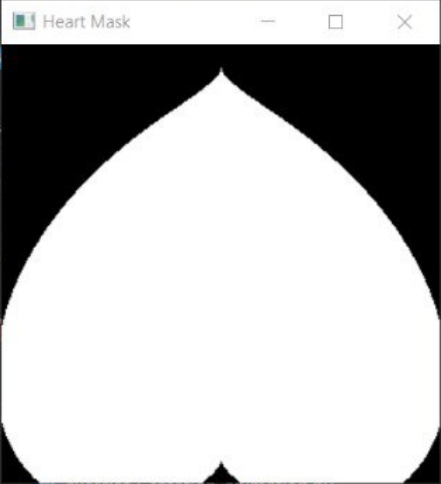
```
Python_Test
exercise3_mask_tool.py exercise4_colour_splash.py exercise5_heart_mask.py splitting
splitting_merging.py
1 # USAGE
2 # python splitting_and_merging.py --image ../images/wave.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import cv2
8
9 # Construct the argument parser and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-i", "--image", required = True,
12                 help = "Path to the image")
13 args = vars(ap.parse_args())
14
15 # Load the image and grab each channel: Red, Green, and Blue
16 # NOTE: OpenCV stores an image as NumPy array with its
17 # channels in reverse order! When we call cv2.split, we are
18 # actually getting the channels as Blue, Green, Red!
19 image = cv2.imread(args["image"])
20 (B, G, R) = cv2.split(image)
21
22 # Show each channel individually
23 cv2.imshow("Red", R)
24 cv2.imshow("Green", G)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

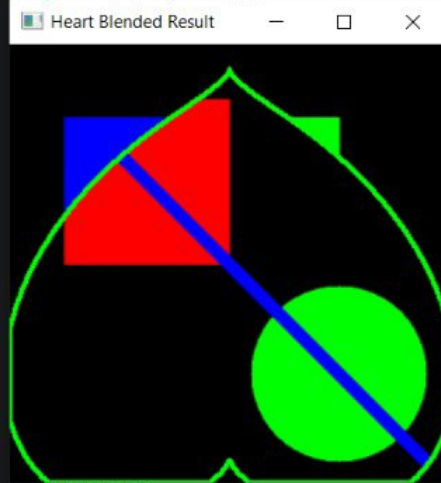
```
C:\Users\iebron-jason\Desktop\Python_Test> python splitting_merging.py --image beach.png
```



Heart Mask



Heart Blended Result



Python\_Test

exercise2\_image\_dissolve.py exercise3\_mask\_tool.py exercise4\_colour\_splash.py exercise5\_heart\_mask.py

exercise5\_heart\_mask.py

```
5 def create_heart_mask(size):
10     for y in range(size):
11         for x in range(size):
21
22     return mask
23
24 # Load images
25 image1 = cv2.imread("test_image.png")
26 image2 = cv2.imread("test_pattern.png")
27 # Resize to same square size
28 size = min(image1.shape[0], image1.shape[1], image2.shape[0],
29 image2.shape[1])
30 size = min(size, 400) # Limit size
31 image1 = cv2.resize(image1, (size, size))
32 image2 = cv2.resize(image2, (size, size))
33 # Create heart mask
34 heart = create_heart_mask(size)
35 heart_inv = cv2.bitwise_not(heart)
36 # Apply mask
37 result = cv2.bitwise_and(image1, image1, mask=heart)
38 result2 = cv2.bitwise_and(image2, image2, mask=heart_inv)
39 final = cv2.add(result, result2)
40 # Draw heart outline
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\jeb\_\Desktop\Python\_Test> python exercise5\_heart\_mask.py

CHAT

Build with Agent

AI responses may be inaccurate  
Generate Agent Instructions to onboard AI onto your codebase.

+ exercise5\_heart\_mask.py

Describe what to build

+ | | Auto |

Type here to search

Record high

17:27 09-06-2026

Python\_Test

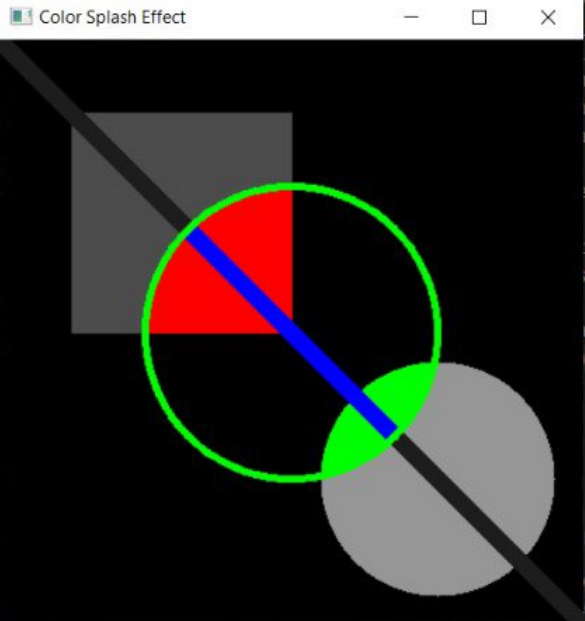
EXPLORER

- PYTHON\_TEST
  - exercise1.py
  - exercise2\_target.py
  - exercise2.py
  - exercise3\_chessboard.py
  - exercise3.py
  - exercise4\_annotati
  - exercise5\_clock.py
  - exercise1\_brightnes
  - exercise1\_thumbnai
  - exercise2\_image\_dis
  - exercise2\_panorama
  - exercise3\_mask\_too
  - exercise3\_rotation
  - exercise4\_colour\_sp
  - exercise4\_smart\_crc
  - exercise5\_image\_ali
  - first\_image.py
  - flipping.py
  - getting\_and\_setting
  - gradient.png
  - house.png
  - imutils.cpython-314
  - imutils.py
  - load\_display\_save.p
  - masked\_result.png
  - my\_masterpiece.png
  - output.bmp
  - output.jpg
  - output.tiff
  - panorama\_prep.png
  - nival\_hacire nu
- OUTLINE
- TIMELINE

exercise4\_colour\_splash.py

```
9 mask = np.zeros(image.shape[:2], dtype="uint8")
10 center = (image.shape[1]//2, image.shape[0]//2)
11 radius = min(image.shape[0], image.shape[1]) // 4
12 cv2.circle(mask, center, radius, 255, -1)
13 # Invert mask: color inside mask, grayscale outside
14 mask_inv = cv2.bitwise_not(mask)
15 # Create a grayscale image of the original image
16 gray_color = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
17 # Apply the mask to the grayscale image
18 and(gray_color, gray_color, mask=mask_inv)
19 # Combine the original image and the masked grayscale image
20 cv2.add(image, image, mask=mask)
21 # Save the result
22 cv2.imwrite("splash.png", splash)
```

Color Splash Effect



TERMINAL

```
Python_Test> python exercise4_colour_splash.py
local\Programs\Python\Python314\python.exe: can't open file 'C:\\Users\\
PS C:\\Users\\jebroon_jason\\Desktop\\Python_Test> python exercise4_colour_splash.py
```

CHAT

Build with Agent

AI responses may be inaccurate  
Generate Agent Instructions to onboard AI  
onto your codebase.

Describe what to build

+ exercise4\_colour\_splash.py

Local Default Approvals

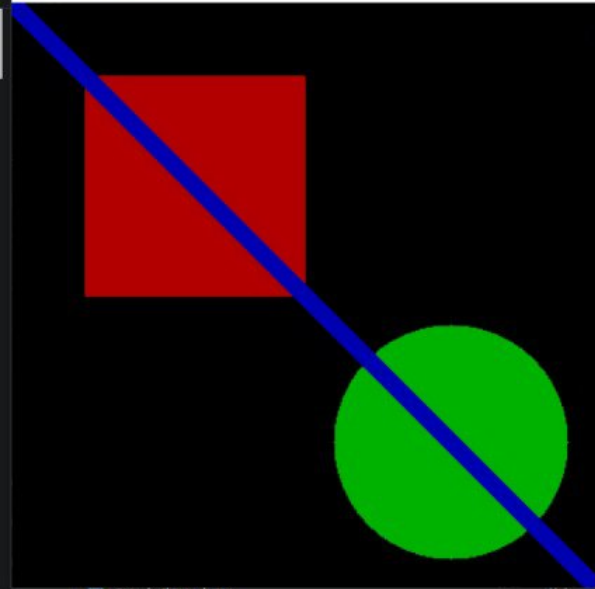
Ln 26, Col 24 Spaces: 4 UTF-8 CRLF Python

17:18 09-06-2026

Type here to search

38°C Partly sunny

Select Region



Python\_Test

ic\_bitwise.py

exercise1\_brightness\_contrast.py

exercise2\_image\_dissolve.py

exercise3\_ma

cise3\_mask\_tool.py

while True:

# Show mask overlay on image

cv2.imshow("Select Region", masked\_vis)

key = cv2.waitKey(10) & 0xFF

if key == ord('s'):

# Apply mask and show result

result = cv2.bitwise\_and(image, image, mask=mask)

cv2.imshow("Masked Result", result)

cv2.imwrite("custom\_mask.png", mask)

cv2.imwrite("masked\_result.png", result)

print("Saved mask and result!")

elif key == ord('r'):

mask = np.zeros(image.shape[:2], dtype="uint8")

print("Mask reset")

elif key == 27: # ESC

break

48 cv2.destroyAllWindows()

Masked Result

ex\_2\_day\_3.py

ex\_3\_day\_3.py

ex\_4\_day\_3.py

exercise1\_house.py

exercise1.py

exercise2\_target.py

exercise2.py

exercise3\_chessboard.py

exercise3.py

exercise4\_annotation.py

exercise5\_clock.py

exercise1\_brightness\_contrast.py

exercise1\_thumbnail\_gallery.py

exercise2\_image\_dissolve.py

exercise2\_panorama\_prep.py

OUTLINE

TIMELINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\jeb\_\jason\Desktop\Python\_Test> python exercise2\_image\_dissolve.py

Creating dissolve transition...

Dissolve complete! Saved as dissolve.mp4

PS C:\Users\jeb\_\jason\Desktop\Python\_Test> python exercise3\_mask\_tool.py

Draw a rectangle on the image to create a mask

Press 's' to save mask, 'r' to reset, ESC to exit

Saved mask and result!

Mask reset

Mask reset

Saved mask and result!

+ powershell

+ powershell

+ powershell

+ powershell

+ powershell

+ powershell

+ powershell

+ powershell

+ python

+ python

+ python

exercise3\_mask\_tool.py

Describe what to build

+ Auto

Local

Default Approvals

Type here to search

38°C Partly sunny

17:15

09-06-2026



The image is a screenshot of a Windows desktop environment. The primary application is Visual Studio Code (VS Code), which is open with a Python project named 'Python\_Test'. The editor displays a file named 'exercise1\_brightness\_contrast.py'. The code in this file is a Python script that uses OpenCV to adjust the brightness and contrast of an image. It includes comments and function calls like 'cv2.add' and 'cv2.imshow'. A terminal window at the bottom of the VS Code interface shows the execution of the script, which has resulted in an 'OverflowError: Python integer -5 out of bounds for uint8'. The left sidebar of VS Code shows a file explorer with various exercise files. On the right side of the screen, there is a 'CHAT' sidebar with a message that says 'Build with Agent' and a prompt to 'Describe what to build'. The Windows taskbar at the bottom shows several open applications, including a web browser, file explorer, and communication tools. The system tray indicates the date and time as 09-06-2026, 17:02.

